PERSONAL NEWS AGREGATOR

Introduction

The **Personalized News Aggregator** is a powerful Python-based solution designed to **automate the collection, processing, categorization, and presentation** of news articles. It pulls news from **popular sources like BBC and CNN**, organizes them into categories using **AI-driven techniques**, and provides a **FastAPI-based API** for seamless access.

This project **addresses the growing need for personalized news retrieval** by leveraging **web scraping, natural language processing (NLP), and AI-powered categorization** to ensure users can efficiently access relevant news. The system not only **extracts and processes news articles in real-time** but also **enhances content understanding** using **machine learning** for automated categorization.

Key benefits of the Personalized News Aggregator:
✔ **Automated News Collection** – No need to manually browse multiple sources.
✔ **AI-Powered Categorization** – Uses **Gemini LLM** for smart classification.
✔ **FastAPI-Based API** – Provides an efficient interface for retrieving and searching articles.
✔ **Summarization of Articles** – Extracts key insights while reducing lengthy text.
✔ **Structured Data Storage** – Ensures efficient organization and retrieval.

Project Overview

The **Personalized News Aggregator** is built as a modular system, consisting of four major components:

1. News Extraction (news_extractor.py)

- **Fetches articles from BBC and CNN** using web scraping.
- Uses **Selenium and BeautifulSoup** to dynamically extract article links and content.
- Processes **HTML content** to extract **titles, summaries, and publication dates**.
- Ensures **duplicate filtering** and **data cleaning** for better accuracy.
- Stores extracted data in a structured format inside news_articles.csv.

2. News Categorization (news_categorization_v2.py)

- **Utilizes Gemini LLM** to categorize articles based on content themes.
- Instead of relying on **static keyword-based categorization**, it uses **AI-driven text understanding**.
- Provides **scalability** as new topics emerge, improving classification accuracy over time.
- **Updates the CSV file** by appending a **category** column to each article.

3. API Aggregation (news_aggregator.py)

- **Implements FastAPI** to create a **RESTful API** for accessing the news database.
- Offers endpoints for:
  ✔ **Fetching all articles** (/articles)
  ✔ **Retrieving an article by ID** (/article-id/{id})
  ✔ **Searching articles by various filters** (/search/{key}/{value})
- Returns responses in **JSON format**, making it easy for integration with frontends or other applications.
- **Provides an interactive API documentation** (/docs) for testing endpoints.

4. Application Runner (start_app.py)

- Manages the execution of the **FastAPI server** using **Uvicorn**.
- Automates the execution of:
  ✔ The **news extractor** to refresh the dataset.
  ✔ The **news categorization module** to update article categories.
- Ensures **seamless deployment and execution** with a single command.

Purpose of the Personalized News Aggregator

The **Personalized News Aggregator** is designed to **streamline news collection, classification, and access** through automation and AI.

- **Efficiently collects news** from multiple sources, reducing the need for manual searching.
- **Summarizes articles** to provide a concise and easy-to-read version of lengthy news reports.
- **Categorizes articles intelligently** to allow users to browse relevant topics effortlessly.
- **Provides a structured API** to enable easy access, integration, and retrieval of news articles.
- **Enhances user experience** by allowing filtering based on category, title, publication date, or source.

By automating **web scraping, data processing, and AI-based categorization**, this system ensures that users receive **high-quality, structured, and relevant news articles in real time**.

Technology Stack

The **Personalized News Aggregator** leverages a combination of **modern web technologies and AI-driven techniques** to efficiently extract, categorize, and serve news articles.

✅ **Backend:** Python, FastAPI – Used for API development and data processing.
✅ **Web Scraping:** BeautifulSoup, Selenium, Requests – Extracts content from **BBC and CNN** dynamically.
✅ **Data Storage:** CSV file (news_articles.csv) – Maintains a structured format for storing extracted news articles.
✅ **AI Categorization:** LlamaIndex with Gemini LLM – Enables **AI-powered article classification**.
✅ **Browser Automation:** Selenium WebDriver – Handles **dynamic content scraping** from websites that require JavaScript execution.
✅ **Frontend:** HTML responses rendered using **FastAPI's built-in response mechanisms** – Allows users to interact with articles directly via a web interface.

Installation Guide

To set up and run the Personalized News Aggregator, follow these steps:

Prerequisites

✔ **Python Version:** Python 3.12.6
✔ **Chrome WebDriver:** Required for Selenium-based web scraping
✔ **Required Libraries:**
Install dependencies using the requirements.txt file:

```
bash
pip install -r requirements.txt
```

Or install them manually:

```
bash
pip install fastapi requests pandas beautifulsoup4 selenium uvicorn
llama_index
```

Running the Application

Follow these steps to **start the News Aggregator**:

1. Start the FastAPI application using Uvicorn

```
bash
python start_app.py
```

This will launch the **news aggregator API** on http://127.0.0.1:8000/.

2. Access the API Documentation

Once the application is running, navigate to:

■ http://127.0.0.1:8000/docs

This provides an **interactive Swagger UI** for testing API endpoints.

Code Structure

The project is structured into multiple modules, each handling a **specific aspect** of the pipeline.

■ Main Components

✅ news_aggregator.py

- Implements the **FastAPI service**.
- Reads articles from news_articles.csv.
- Exposes API **endpoints for retrieving and searching articles**.

✅ news_extractor.py

- Scrapes **BBC and CNN** for the latest news.
- Uses **Selenium and BeautifulSoup** for extracting content.
- Saves structured data into news_articles.csv.

✅ news_categorization_v2.py

- Uses **Gemini LLM** to categorize articles based on their content.
- Adds an AI-generated category column to news_articles.csv.

✅ start_app.py

- Automates **running the extractor and categorization** before starting the API.
- Runs the **FastAPI server** with Uvicorn.

✅ requirements.txt

- Lists all **dependencies** required for the project.

Overview of Code Files

1. news_aggregator.py (API Layer)

- The core **API service** of the project.
- Handles requests for **fetching articles, searching news, and retrieving articles by ID**.
- Renders **HTML responses** for better interaction.

2. news_extractor.py (News Scraper)

- **Extracts** news articles from **BBC and CNN**.
- Uses **Selenium and BeautifulSoup** for scraping dynamically loaded content.
- Saves the extracted data in **CSV format (**news_articles.csv**)**.

3. news_categorization_v2.py (AI Categorization)

- Uses **Gemini LLM** to classify articles **intelligently**.
- Assigns categories such as **politics, technology, sports, and more**.
- Updates the **CSV file with AI-predicted categories**.

4. start_app.py (Application Runner)

- Automates the **news extraction and categorization process** before launching the API.
- Ensures that **data is up to date** each time the server starts.

Description of Key Functions

The **Personalized News Aggregator** consists of multiple functions that automate the process of **fetching, processing, storing, and categorizing news articles**.

Fetching Articles from News Sources (BBC, CNN)

The first step in the pipeline is to **extract URLs** of relevant news articles from BBC and CNN.

Code Snippet (BBC News Extraction)

```
def func__ext_urls__bbc():
    url__bbc = 'https://www.bbc.com/news'
    soup = BeautifulSoup(requests.get(url__bbc).text, 'html.parser')
    news__urls = list(set([
        f"https://www.bbc.com{item['href']}"
        for item in soup.find_all('a', href=True)
        if item['href'].startswith('/news/articles/')
    ]))[:20]
    return news__urls
```

✓ **Purpose:** Extracts URLs of BBC news articles that match the **/news/articles/** pattern.

✓ **Usage:** This function **scrapes and returns the top 20 articles** available on the BBC homepage.

Preprocessing Article Summaries

To provide a **concise and meaningful summary**, the text extracted from an article needs to be **cleaned and structured properly**.

Code Snippet (Text Processing & Summarization)

```
def preprocess_summary(text, max_sentences=4):
    soup = BeautifulSoup(text, 'html.parser')
    paragraphs = [p.text.strip() for p in soup.find_all('p') if
p.text.strip()]
    combined_text = ' '.join(paragraphs)

    # Remove extra spaces and newlines
    combined_text = re.sub(r'\s+', ' ', combined_text).strip()

    # Split into sentences and limit to max_sentences
    sentences = re.split(r'(?<!\w\.\w.)(?<![A-Z][a-z]\.)(?<=\.|\?)\s',
combined_text)
    summary = ' '.join(sentences[:max_sentences])

    return summary
```

✓ **Purpose:** Cleans raw article content and generates a structured summary **(limited to 4 sentences)**.

✅ **Usage:** Reduces **unnecessary text** and ensures that users receive **key insights** without reading the full article.

Extracting Publication Dates

Publication dates are **crucial** for organizing news articles by timeline. Since some websites display **relative timestamps (e.g., "2 days ago")**, additional processing is required.

Code Snippet (Extracting Dates from BBC)

```
def func_ext_news_bbc(url):
    driver.get(url)
    page_source = driver.page_source
    soup = BeautifulSoup(page_source, 'html.parser')
    published_datetime = re.findall(r'<time class="sc-2b5e3b35-2 fkLXLN">(.*?)</time>', str(page_source))[-1]

    # Process the extracted date to generate a standard publication date format
    return published_datetime
```

✅ **Purpose:** Extracts the **publication date** from the HTML structure of BBC news articles.

✅ **Usage:** Converts **relative timestamps (e.g., "2 days ago") into an actual date** format for accurate storage.

Storing and Managing Data in CSV

Once the news articles are **scraped and processed**, they need to be **saved in a structured format** for easy retrieval. The **CSV file (**news_articles.csv**)** serves as the project's **primary data storage**.

Code Snippet (Generating CSV)

```
df_csv = pd.DataFrame(dct_df).drop_duplicates(subset=['title', 'summary', 'url']).reset_index(drop=True)
df_csv.to_csv('news_articles.csv', index=False)
```

✅ **Purpose:** Saves extracted articles into a **CSV file**, ensuring **no duplicates** are stored.

✅ **Usage:** This step **preserves the data** and makes it accessible for later use.

Reading Data from CSV for API Access

Once the articles are stored, they need to be **read back into the system** for retrieval through the **FastAPI interface**.

Code Snippet (Reading CSV)

```
df_dct = pd.read_csv('news_articles.csv').to_dict(orient='index')
```

✅ **Purpose:** Loads the stored CSV file into a **dictionary format** for **fast lookups and API responses**.

✅ **Usage:** Ensures that the API can **quickly fetch articles** based on user queries.

Summary of Functional Improvements

Automated Web Scraping – Fetches real-time articles from **BBC and CNN**.
AI-Powered Summarization – Extracts **key insights** from each article.
Publication Date Standardization – Converts **relative timestamps into proper dates**.
Structured Data Storage – Saves **deduplicated** articles in a CSV file.
Optimized API Access – Loads stored articles **efficiently for fast retrieval**.

API Endpoints

The **FastAPI backend** provides multiple endpoints for users to **retrieve, search, and filter news articles**.

Base URL: `http://127.0.0.1:8000/`

---

1. Root (/)

Displays a **simple HTML page** with links to key API routes.

Code Snippet

```
@app.get("/")def read_root():
    html__index = """
    <html>
        <head>
            <title>News Aggregator</title>
            <style>
                body { text-align: center; font-family: Arial, sans-
serif; }
            </style>
        </head>
        <body>
            <h1>News Aggregator :: Index</h1>
            <br>
            <a href="/articles">View All Articles</a><br>
            <a href="/article-id">Search by Article ID</a><br>
            <a href="/search">Search Articles</a><br>
            <a href="/docs" target="_blank">API Documentation</a>
        </body>
    </html>
    """
    return HTMLResponse(content=html__index)
```

✅ **Purpose:** Serves as a homepage for users to navigate the available routes.

✅ **Usage:** Visit http://127.0.0.1:8000/ in a web browser to explore the API.

---

2. Get All Articles (/articles)

Retrieves **all news articles** stored in the system in **JSON format**.

Code Snippet

```
@app.get("/articles")def read_articles():
    return JSONResponse(content=json.dumps(df_dct, indent=4))
```

✅ **Purpose:** Returns a **JSON response** containing all articles.

✅ **Example Response:**

```
json
{
    "0": {
        "title": "BBC News - Politics",
        "summary": "The government has announced new policies...",
```

```
        "publication_date": "2024-02-05",
        "source": "BBC",
        "url": "https://www.bbc.com/news/example"
    },
    "1": {
        "title": "CNN - Technology Breakthrough",
        "summary": "New AI models are changing the industry...",
        "publication_date": "2024-02-06",
        "source": "CNN",
        "url": "https://www.cnn.com/tech/example"
    }}
```

✅ **Usage:**
Open http://127.0.0.1:8000/articles in a browser or use:

```
curl -X GET "http://127.0.0.1:8000/articles"
```

---

3. Retrieve an Article by ID (/article-id/{id})

Fetches a **specific article** using its **unique ID**.

Code Snippet

```
@app.get("/article-id/{id}")def read_article_id(id: int):
    if id in df_dct.keys():
        return JSONResponse(content=df_dct[id])
    else:
        return JSONResponse(content={"error": "Article not found"})
```

✅ **Purpose:** Allows users to **retrieve a single article** based on its **ID**.

✅ **Example Request:**
Fetch the article with ID 1:

```
curl -X GET "http://127.0.0.1:8000/article-id/1"
```

✅ **Example Response:**

```
{
    "title": "CNN - Technology Breakthrough",
    "summary": "New AI models are changing the industry...",
    "publication_date": "2024-02-06",
    "source": "CNN",
    "url": "https://www.cnn.com/tech/example"}
```

✅ **Error Handling:**
If an **invalid article ID** is provided, it returns:

```
json
{
    "error": "Article not found"}
```

---

4. Search Articles (/search/{key}/{value})

Allows users to **filter articles** based on specific fields like **title, summary, date, source, or category**.

Code Snippet

```
@app.get("/search/{key}/{value}")def search_article_keyword(key: str, value: str):
    filtered_data = {k: v for k, v in df_dct.items() if value.lower() in v.get(key, "").lower()}

    if filtered_data:
        return JSONResponse(content=jsonable_encoder(filtered_data))
    else:
        return JSONResponse(content={"error": "No articles found matching the criteria"})
```

✅ **Purpose:** Enables **custom search queries** based on **title, summary, publication date, source, or category**.

✅ **Example Requests:**
Search articles containing "AI" in the title:

```
curl -X GET "http://127.0.0.1:8000/search/title/AI"
```

Search for all articles published on "2024-02-05":

```
curl -X GET "http://127.0.0.1:8000/search/publication_date/2024-02-05"
```

✅ **Example Response:**

```
{
    "0": {
        "title": "CNN - Technology Breakthrough",
        "summary": "New AI models are changing the industry...",
        "publication_date": "2024-02-06",
        "source": "CNN",
```

```
        "url": "https://www.cnn.com/tech/example"
    }}
```

✅ **Error Handling:**
If **no matching articles** are found, it returns:

```
{
    "error": "No articles found matching the criteria"}
```

Summary of API Features

Endpoint	Function	Example Usage
/	Displays an **HTML page** with links to API routes.	Open in a browser
/articles	Returns **all news articles** in **JSON format**.	GET /articles
/article-id/{id}	Fetches a **specific article by ID**.	GET /article-id/1
/search/{key}/{value}	**Filters articles** based on **title, date, source, etc.**.	GET /search/title/AI

Detailed Explanation of Functions

The **Personalized News Aggregator** consists of several key functions responsible for **fetching, processing, categorizing, and storing news articles**.

---

1. News Article Scraping (BBC and CNN)

The first step is to **extract news articles** from trusted sources like **BBC and CNN** using **web scraping** techniques.

BBC News Scraping

✅ func__ext_urls__bbc()

- **Purpose:** Fetches article URLs from **BBC News** homepage.
- **Method:** Uses **BeautifulSoup** to parse and extract relevant news links.

Code Snippet (Extracting BBC URLs)

```
def func__ext_urls__bbc():
    url__bbc = 'https://www.bbc.com/news'
    soup = BeautifulSoup(requests.get(url__bbc).text, 'html.parser')
    news__urls = list(set([
        f"https://www.bbc.com{item['href']}"
        for item in soup.find_all('a', href=True)
        if item['href'].startswith('/news/articles/')
    ]))[:20]
    return news__urls
```

✓ func__ext_news__bbc()

- **Purpose:** Fetches and extracts article **title, summary, publication date, and source**.
- **Method:** Uses **Selenium with a headless Chrome browser** to process dynamically loaded content.

Code Snippet (Extracting BBC News Data)

```
def func__ext_news__bbc(url):
    driver.get(url)
    page_source = driver.page_source
    soup = BeautifulSoup(page_source, 'html.parser')

    title = soup.title.string if soup.title else "No Title"
    summary = preprocess_summary(page_source)
    publication_date = datetime.strptime(soup.find('time')['datetime'], "%Y-%m-%dT%H:%M:%S.%fZ").strftime("%Y-%m-%d")

    return {"title": title, "summary": summary, "publication_date": publication_date, "source": "BBC", "url": url}
```

CNN News Scraping

✓ func__ext_urls__cnn()

- **Purpose:** Fetches article URLs from **CNN** homepage.
- **Method:** Uses **BeautifulSoup** to parse **CNN homepage** and extract relevant links.

Code Snippet (Extracting CNN URLs)

```
def func__ext_urls__cnn():
    url__cnn = "https://www.cnn.com"
    soup = BeautifulSoup(requests.get(url__cnn).text, 'html.parser')
    news__urls = list(set([
        url__cnn + link['href'] if link['href'].startswith('/') else
link['href']
        for link in soup.find_all('a', href=True)
        if f'/{datetime.now().year}/' in link['href'] and 'video' not
in link['href']
    ]))[:20]
    return news__urls
```

✅ func__ext_news__cnn()

- **Purpose:** Extracts **CNN article details** including **title, summary, publication date, and source**.
- **Method:** Uses **BeautifulSoup** to parse static HTML content.

Code Snippet (Extracting CNN News Data)

```
def func__ext_news__cnn(url):
    soup = BeautifulSoup(requests.get(url).content, 'html.parser')
    title = soup.title.string if soup.title else "No Title"
    summary = preprocess_summary(soup.prettify())
    publication_date = "-".join(url.split('/')[3:6])

    return {"title": title, "summary": summary, "publication_date":
publication_date, "source": "CNN", "url": url}
```

---

2. Data Preprocessing and Categorization

After extracting the news articles, they **undergo preprocessing** to remove **HTML elements, extra spaces, and unnecessary text** before categorization.

Preprocessing Summaries

✅ preprocess_summary()

- **Purpose:** Cleans and structures article summaries.
- **Method:** Removes **HTML tags, extra spaces, and limits summary length** to **4 sentences**.

Code Snippet (Summary Preprocessing)

```
def preprocess_summary(text, max_sentences=4):
    soup = BeautifulSoup(text, 'html.parser')
    paragraphs = [p.text.strip() for p in soup.find_all('p') if p.text.strip()]
    combined_text = ' '.join(paragraphs)
    combined_text = re.sub(r'\s+', ' ', combined_text).strip()
    sentences = re.split(r'(?<!\w\.\w.)(?<![A-Z][a-z]\.)(?<=\.|\?)\s', combined_text)
    summary = ' '.join(sentences[:max_sentences])

    return summary
```

✅ **Usage:** Reduces long texts while **retaining essential information**.

AI-Powered Categorization

The system categorizes articles using **Gemini LLM**, providing **dynamic, AI-driven classification**.

✅ categorize_with_llm()

- **Purpose:** Uses **Gemini AI** to assign categories based on **article content**.
- **Method:** Generates a structured request with all summaries and processes them in a single LLM call.

Code Snippet (AI-Based Categorization)

```
def categorize_with_llm(df):
    summaries_text = "\n".join([f"{i+1}. {summary}" for i, summary in enumerate(df['summary'])])

    query_str = (
        "Categorize these articles:\n"
        f"{summaries_text}\n\n"
        "Format response as:\n"
        "1. category_name\n2. category_name\n..."
    )

    cat_response = category_agent.complete(query_str).text.strip()
    category_list = [line.split(". ")[1].lower() for line in cat_response.split("\n") if ". " in line]
```

```
    if len(category_list) == len(df):
        df['category'] = category_list
    return df
```

✅ **Advantages of AI Categorization:**
✔ No need for **predefined keyword matching**.
✔ More **flexibility and accuracy** in assigning categories.

---

3. Data Storage (CSV File)

After processing, the extracted data is stored in a **CSV file (**news_articles.csv**)**, ensuring structured storage and easy retrieval.

✅ **CSV File Contains the Following Columns:**

Column	Description
title	Title of the news article
summary	Shortened version of the article
publication_date	Date the article was published
source	News source (BBC or CNN)
url	Link to the full article
category	AI-predicted category

✅ **Saving Data to CSV**

```
df_csv = pd.DataFrame(dct_df).drop_duplicates(subset=['title', 'summary', 'url']).reset_index(drop=True)
df_csv.to_csv('news_articles.csv', index=False)
```

✅ **Reading Data from CSV**

```
df_dct = pd.read_csv('news_articles.csv').to_dict(orient='index')
```

Code Execution

The **Personalized News Aggregator** follows a structured execution pipeline that consists of:
✅ **News Scraping (BBC & CNN)** – Extracts URLs and article content.
✅ **Data Preprocessing** – Cleans and summarizes articles.

✅ **AI-Powered Categorization** – Uses **Gemini LLM** to categorize news.
✅ **Data Storage** – Saves extracted news in news_articles.csv.

---

1. News Article Scraping (BBC and CNN)

This step fetches articles from **BBC and CNN**, extracts relevant information, and processes the content.

1.1. Extracting BBC News URLs

✅ **Function:** func__ext_urls__bbc()
✅ **Purpose:** Extracts **top 20 BBC news article URLs**.

Code Snippet (BBC News URL Extraction)

```
def func__ext_urls__bbc():
    url__bbc = 'https://www.bbc.com/news'
    soup = BeautifulSoup(requests.get(url__bbc).text, 'html.parser')
    news__urls = list(set([
        f"https://www.bbc.com{item['href']}"
        for item in soup.find_all('a', href=True)
        if item['href'].startswith('/news/articles/')
    ]))[:20]
    return news__urls
```

✅ **Example Output:**

```
[
    "https://www.bbc.com/news/world-news/65105555",
    "https://www.bbc.com/news/uk-65107735",
    "https://www.bbc.com/news/business-65106789",
    "https://www.bbc.com/news/entertainment-arts-65107649",
    "https://www.bbc.com/news/technology-65107577"]
```

1.2. Extracting CNN News URLs

✅ **Function:** func__ext_urls__cnn()
✅ **Purpose:** Extracts **top 20 CNN news article URLs** from the current year.

Code Snippet (CNN News URL Extraction)

```
def func__ext_urls__cnn():
    url__cnn = "https://www.cnn.com"
    soup = BeautifulSoup(requests.get(url__cnn).text, 'html.parser')
    news__urls = list(set([
        url__cnn + link['href'] if link['href'].startswith('/') else
link['href']
        for link in soup.find_all('a', href=True)
        if f'/{datetime.now().year}/' in link['href'] and 'video' not
in link['href']
    ]))[:20]
    return news__urls
```

✅ **Example Output:**

```
[
    "https://www.cnn.com/world/live-news/russia-ukraine-war-news",
    "https://www.cnn.com/politics/news/trump-indictment-new-york",
    "https://www.cnn.com/us/live-news/california-wildfires-2024",
    "https://www.cnn.com/business/news/us-economy-recession-outlook",
    "https://www.cnn.com/health/live-news/coronavirus-pandemic-
latest-updates"]
```

1.3. Extracting News Content from BBC

✅ **Function:** func__ext_news__bbc(url)
✅ **Purpose:** Extracts **title, summary, publication date, and source** from BBC articles.

Code Snippet (BBC News Extraction)

```
def func__ext_news__bbc(url):
    try:
        driver.get(url)
        page_source = driver.page_source
        soup = BeautifulSoup(page_source, 'html.parser')

        title = soup.title.string if soup.title else "No Title"
        summary = preprocess_summary(page_source)
        publication_date = datetime.strptime(
            soup.find('time')['datetime'], "%Y-%m-%dT%H:%M:%S.%fZ"
        ).strftime("%Y-%m-%d")
```

```
        return {'title': title, 'summary': summary,
'publication_date': publication_date, 'source': 'BBC', 'url': url}

    except Exception as e:
        print(f"Error processing BBC article at {url}: {e}")
        return None
```

✅ **Example Output:**

```
{

    "title": "BBC News - UK",
    "summary": "The UK government has announced plans to increase
spending on defence and security.",
    "publication_date": "2024-09-14",
    "source": "BBC",
    "url": "https://www.bbc.com/news/uk-65107735"}
```

1.4. Extracting News Content from CNN

✅ **Function:** func__ext_news__cnn(url)
✅ **Purpose:** Extracts **title, summary, publication date, and source** from CNN articles.

Code Snippet (CNN News Extraction)

```
def func__ext_news__cnn(url):
    try:
        soup = BeautifulSoup(requests.get(url).content, 'html.parser')
        title = soup.title.string if soup.title else "No Title"
        summary = preprocess_summary(soup.prettify())
        publication_date = "-".join(url.split('/')[3:6])

        return {'title': title, 'summary': summary,
'publication_date': publication_date, 'source': 'CNN', 'url': url}

    except Exception as e:
        print(f"Error processing CNN article at {url}: {e}")
        return None
```

✅ **Example Output:**


```
{
    "title": "CNN: Your World, Your News",
    "summary": "CNN is a global news platform that provides breaking news, live coverage, and in-depth analysis.",
    "publication_date": "2024-09-14",
    "source": "CNN",
    "url": "https://www.cnn.com/"}
```


---

2. Data Preprocessing and Categorization

2.1. Preprocessing Article Summaries

✅ **Function:** preprocess_summary()
✅ **Purpose:** Cleans and structures article summaries.

Code Snippet (Summary Preprocessing)

```
def preprocess_summary(text, max_sentences=4):
    soup = BeautifulSoup(text, 'html.parser')
    paragraphs = [p.text.strip() for p in soup.find_all('p') if p.text.strip()]
    combined_text = ' '.join(paragraphs)
    combined_text = re.sub(r'\s+', ' ', combined_text).strip()
    sentences = re.split(r'(?<!\w\.\w.)(?<![A-Z][a-z]\.)(?<=\.|\?)\s', combined_text)
    summary = ' '.join(sentences[:max_sentences])

    return summary
```

✅ **Example Output:**


```
"The UK government has announced plans to increase spending on defence and security."
```

2.2. AI-Based Categorization

✅ **Function:** categorize_with_llm()
✅ **Purpose:** Uses **Gemini LLM** to assign categories based on article content.

✅ **Example Output:**


```
"politics"
```


---

3. Data Storage (CSV Format)

✅ **Function:** Stores **scraped and processed news** in a **CSV file**.

Code Snippet (Saving to CSV)

```
df_csv = pd.DataFrame(dct_df).drop_duplicates(subset=['title', 'summary', 'url']).reset_index(drop=True)
df_csv.to_csv('news_articles.csv', index=False)
```

✅ **Example Output:**
A CSV file (news_articles.csv**) is created with columns:**
✔ **title** | ✔ **summary** | ✔ **publication_date** | ✔ **source** | ✔ **url** | ✔ **category**

Web Interface

The **Personalized News Aggregator** includes an interactive **web interface** for users to navigate, search, and retrieve news articles through **HTML pages and JSON responses**.

Key Features:

✔ **Navigation through HTML pages** (/) – Provides easy access to API functionalities.
✔ **Interactive Article Search** (/article-id) – Users can input an **article ID** to fetch details.
✔ **News Listing** (/articles) – Displays all articles in a structured **JSON format**.
✔ **Keyword-Based Search** (/search/{key}/{value}) – Allows filtering based on title, summary, date, source, or category.

---

1. Homepage (/)

Displays a navigation page with links to key API features:

- **View All Articles** (/articles)
- **Search by Article ID** (/article-id)
- **Search Articles** (/search)
- **API Documentation** (/docs)

✅ **Purpose:** Helps users explore available API functionalities easily.
✅ **Example Usage:** Open http://127.0.0.1:8000/ in a web browser.

---

2. Articles Page (/articles)

Fetches all news articles in JSON format.

✅ **Purpose:** Provides a structured list of articles with **titles, summaries, publication dates, and sources**.
✅ **Example Usage:** Access http://127.0.0.1:8000/articles to view all available news.
✅ **Output:** JSON response with all articles stored in news_articles.csv.

---

3. Article Lookup by ID (/article-id)

Interactive search interface for retrieving a **specific article by its ID**.

✅ **Purpose:** Allows users to **enter an article ID** and fetch relevant details dynamically.
✅ **Example Usage:** Visit http://127.0.0.1:8000/article-id, enter an **article ID**, and get details.
✅ **Output:** JSON response containing **title, summary, publication date, source, and URL**.

---

4. Retrieve an Article by ID (/article-id/{id})

Fetches a single article in JSON format based on ID.

✅ **Purpose:** Quickly retrieve **detailed news content** for a specific article.
✅ **Example Usage:** http://127.0.0.1:8000/article-id/1
✅ **Output:** Returns article details if **ID exists**, otherwise returns "error": "Article not found".

---

5. Search Articles (/search/{key}/{value})

Allows searching articles based on different fields.

✅ **Available Search Fields:**

- **title** → Search for specific words in the article title.
- **summary** → Filter articles based on key terms in the summary.
- **publication_date** → Find articles published on a specific date.
- **source** → Retrieve news only from **BBC or CNN**.
- **category** → Fetch articles belonging to a particular **AI-assigned category**.

✅ **Example Searches:**

- Find **AI-related articles**: http://127.0.0.1:8000/search/title/AI
- Find news **from CNN**: http://127.0.0.1:8000/search/source/CNN
- Get articles **published on a specific date**: http://127.0.0.1:8000/search/publication_date/2024-02-05

✅ **Output:** JSON response with **matching articles.**

Usage Examples

Fetching All Articles

Retrieve all news articles stored in the system.
✅ **URL:** http://127.0.0.1:8000/articles
✅ **Example Response:**


```
{
    "0": {
        "title": "BBC News - Politics",
        "summary": "The government has announced new policies...",
        "publication_date": "2024-02-05",
        "source": "BBC",
        "url": "https://www.bbc.com/news/example"
    }}
```

Searching for Articles by Keyword

Find articles based on **title, summary, date, source, or category**.

✅ **URL Format:**

```
http://127.0.0.1:8000/search/{field}/{value}
```

✅ **Example Searches:**

- Find **BBC news**: http://127.0.0.1:8000/search/title/BBC
- Find **Technology-related articles**: http://127.0.0.1:8000/search/category/technology
- Find articles **published on a specific date**: http://127.0.0.1:8000/search/publication_date/2024-02-05

Retrieving an Article by ID

Fetch a specific **news article using its unique ID**.
✅ **URL Format:**

```
http://127.0.0.1:8000/article-id/{id}
```

✅ **Example:** Retrieve **Article ID 1**
http://127.0.0.1:8000/article-id/1

✅ **Example Response:**

```
{
    "title": "CNN - Technology Breakthrough",
    "summary": "New AI models are changing the industry...",
    "publication_date": "2024-02-06",
    "source": "CNN",
    "url": "https://www.cnn.com/tech/example"}
```

Accessing API Documentation

Explore and test API endpoints **via interactive Swagger UI**.
✅ **URL:** http://127.0.0.1:8000/docs

Deployment

Running Locally with Uvicorn

Start the FastAPI server for local testing.
✅ **Command:**

```
uvicorn news_aggregator:app --reload
```

✅ **Access API at:** http://127.0.0.1:8000/

Deployment to Production

The project is **hosted on GitHub** for easy access and future enhancements.
✅ **GitHub Repository:**

https://github.com/praneeth4387/personalized_news_aggregator_with_llms

---

Best Practices

API Design Guidelines

✅ **Maintain Consistency** – Use intuitive and well-structured API endpoints:

- /articles → Fetch all articles
- /article-id/{id} → Retrieve a specific article
- /search/{key}/{value} → Search for articles by keyword

✅ **Follow RESTful Standards** – Ensure predictable **request-response patterns**.

Efficient Data Handling

✅ **Lightweight Storage** – Uses **CSV files** for easy scalability and **faster access**.
✅ **Avoids Unnecessary Computation** – **Preprocesses summaries** before storing them.

Error Handling & Robustness

✅ **Meaningful Error Messages** – Ensures users get clear responses:

- **Invalid article ID:** { "error": "Article not found" }
- **No search results found:** { "error": "No matching articles" }

✅ **Handles Web Scraping Failures** – Skips faulty articles and logs errors.

Future Enhancements

Adding More News Sources

Expand to Reuters, Al Jazeera, and other global sources to increase content diversity.

Improving Search Functionality

Implement **fuzzy search** (typo tolerance) and **advanced filtering** by:
✔ **Date Range** (e.g., last 7 days, last month)
✔ **Multiple Categories** (e.g., Politics + Technology)

Expanding to Other Data Formats

Support alternative storage formats like:
✔ **JSON/XML Databases** – Structured and API-friendly.
✔ **SQL Database (PostgreSQL/MySQL)** – For better scalability.

---

17. Conclusion

The **Personalized News Aggregator** successfully:
✅ **Scrapes & processes news articles** from multiple sources.
✅ **Uses AI-powered categorization** for better content organization.
✅ **Provides a user-friendly FastAPI service** for seamless access.

Future improvements will focus on:
✔ Expanding to **more news sources**.
✔ Enhancing **search & filtering options**.
✔ Supporting **new data storage formats** for scalability.